

MASTERING SNOWFLAKE

From Beginner to Expert



- ✓ Data Warehousing & Performance Tuning
- ✓ Security & Data Governance
- ✓ ETL & Automation Pipelines
- ✓ UDFs & Stored Procedures
- ✓ Data Sharing & Replication
- ✓ Cost Management & Monitoring



BECOME A SNOWFLAKE EXPERT!



INDEX

Chapter 1: Foundations of Cloud & Modern Data Warehousing

- 1.1 Introduction to Cloud Computing
 - 1.2 Evolution of Data Warehouses
 - 1.3 Limitations of Traditional Data Warehouses
 - 1.4 Need for Cloud Data Platform
 - 1.5 Introduction to Snowflake
 - 1.6 Snowflake Editions & Pricing Model
 - 1.7 Key Advantages of Snowflake
 - 1.8 Real-World Use Cases
 - 1.9 Interview Focus Points
 - 1.10 Exercises
-

Chapter 2: Snowflake Architecture – The 3-Layer Design

- 2.1 Overview of Snowflake Architecture
 - 2.2 Storage Layer (Database Storage Architecture)
 - 2.3 Compute Layer (Virtual Warehouses)
 - 2.4 Service Layer (Cloud Services Layer)
 - 2.5 Separation of Storage and Compute
 - 2.6 Multi-Cluster Warehouses
 - 2.7 Concurrency Handling
 - 2.8 Architecture-Based Advantages
 - 2.9 Real-World Scenario Analysis
 - 2.10 Exercises
-

Chapter 3: Micro-Partitions & Data Clustering

- 3.1 What are Micro-Partitions?
- 3.2 How Snowflake Stores Data Internally
- 3.3 Columnar Storage Concept
- 3.4 Metadata Storage & Pruning
- 3.5 Natural Clustering
- 3.6 Clustering Keys
- 3.7 Automatic Clustering (Re-Clustering)
- 3.8 Performance Optimization Strategies
- 3.9 Practical Use Cases
- 3.10 Exercises

Chapter 4: Snowflake Caching Mechanism

- 4.1 Importance of Caching
 - 4.2 Result Cache
 - 4.3 Metadata Cache
 - 4.4 SSD (Local Disk) Cache
 - 4.5 Remote Disk Cache
 - 4.6 Cache Invalidation Rules
 - 4.7 Performance Tuning Using Cache
 - 4.8 Monitoring Query Performance
 - 4.9 Real-World Optimization Case Study
 - 4.10 Exercises
-

Chapter 5: Security & Access Control

- 5.1 Security Overview in Snowflake
 - 5.2 Role-Based Access Control (RBAC)
 - 5.3 Discretionary Access Control (DAC)
 - 5.4 Users, Roles & Privileges
 - 5.5 Role Hierarchy Design
 - 5.6 Data Masking (DDM)
 - 5.7 Row Access Policies (RAP)
 - 5.8 Tag-Based Masking Policies
 - 5.9 Classification & Sensitive Data Handling
 - 5.10 Best Practices for Enterprise Security
 - 5.11 Exercises
-

Chapter 6: Continuous Data Protection

- 6.1 Time Travel
 - 6.2 Data Retention Period
 - 6.3 Undrop Objects
 - 6.4 Fail-Safe
 - 6.5 Zero-Copy Cloning
 - 6.6 Backup & Recovery Strategy
 - 6.7 Enterprise Data Governance
 - 6.8 Practical Scenarios
 - 6.9 Exercises
-

Chapter 7: Data Loading & Unloading

- 7.1 Overview of Data Ingestion
 - 7.2 Bulk Loading Concepts
 - 7.3 SnowSQL Installation & Configuration
 - 7.4 CSV Data Loading
 - 7.5 JSON Data Loading
 - 7.6 Stages (Internal & External)
 - 7.7 COPY INTO Command
 - 7.8 Data Validation & Error Handling
 - 7.9 Unloading Data to CSV
 - 7.10 Unloading Data to JSON
 - 7.11 Performance Best Practices
 - 7.12 Exercises
-

Chapter 8: Snowflake Objects & Programming

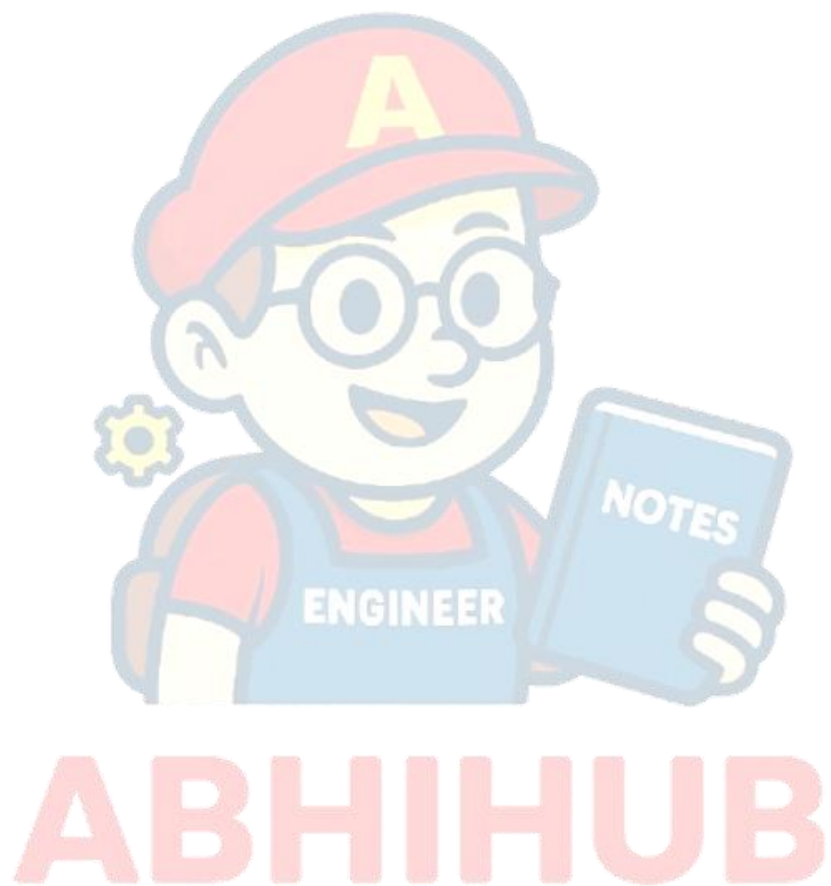
- 8.1 Tables & Views
 - 8.2 Streams
 - 8.3 Tasks
 - 8.4 Pipes (Snowpipe)
 - 8.5 User Defined Functions (UDF)
 - 8.6 Stored Procedures
 - 8.7 External Functions
 - 8.8 External Tokenization
 - 8.9 Dynamic Tables
 - 8.10 Hybrid Tables
 - 8.11 Practical ETL Implementation
 - 8.12 Exercises
-

Chapter 9: Continuous Data Pipeline & Data Sharing

- 9.1 Continuous Data Pipeline Architecture
 - 9.2 Streams + Tasks Automation
 - 9.3 Snowpipe Working Mechanism
 - 9.4 Data Sharing Concepts
 - 9.5 Secure Data Sharing
 - 9.6 Cross-Region Replication
 - 9.7 Data Exchange
 - 9.8 Real-Time Ingestion Architecture
 - 9.9 Enterprise Use Cases
 - 9.10 Exercises
-

Chapter 10: Monitoring, Governance & Advanced Topics

- 10.1 Resource Monitor
- 10.2 Cost Optimization Strategies
- 10.3 Access History
- 10.4 Object Dependencies
- 10.5 Search Optimization Service
- 10.6 Tagging Framework
- 10.7 Governance Architecture
- 10.8 Performance Troubleshooting
- 10.9 Capstone Project – End-to-End Implementation
- 10.10 Interview Preparation Guide



CHAPTER 1

FOUNDATION OF CLOUD DATA WAREHOUSE & SNOWFLAKE

1.1 Introduction to Data Warehousing

1.1.1 What is a Data Warehouse?

A **Data Warehouse (DW)** is a centralized system designed to store large volumes of structured data for analysis and reporting.

Unlike OLTP databases (used for transactions), a Data Warehouse is used for:

- Business Intelligence (BI)
- Reporting
- Data Analytics
- Historical trend analysis

Example

Banking system:

- Transaction DB → handles daily transactions
- Data Warehouse → analyzes monthly spending trends

1.1.2 OLTP vs OLAP

Feature	OLTP	OLAP
Purpose	Transactions	Analytics
Example	ATM withdrawal	Monthly revenue report
Data Size	Small	Very large
Queries	Short & frequent	Complex & heavy

Snowflake is primarily an **OLAP system**.

1.2 Evolution of Data Warehouses

Let us travel through time.

1.2.1 Traditional On-Premise Systems

Earlier systems were installed in company data centers.

Major vendors:

- Oracle Corporation
- IBM
- Microsoft

Architecture Characteristics

- Fixed hardware servers
- Compute + Storage tightly coupled
- Manual scaling
- Expensive maintenance

Problem

If storage increased → buy new hardware

If compute increased → buy bigger server

This caused:

- High capital expenditure (CAPEX)
- Long setup time
- Resource wastage

1.2.2 Need for Change

Businesses required:

- Elastic scaling
- Faster provisioning
- Pay-as-you-use model
- Global accessibility

This led to **Cloud Computing**.

1.3 Cloud Computing Basics

1.3.1 What is Cloud Computing?

Cloud computing provides on-demand computing services over the internet.

Major Providers:

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform

1.3.2 Service Models

IaaS (Infrastructure as a Service)

- Virtual machines
- Storage
- Networking

PaaS (Platform as a Service)

- Managed database
- Application platform

SaaS (Software as a Service)

- Fully managed software

Snowflake operates as **SaaS**.

You do not manage:

- Hardware
- OS
- Database patching
- Infrastructure

1.3.3 Key Cloud Characteristics

1. On-demand provisioning
2. Elastic scaling
3. Measured service (pay for what you use)
4. Resource pooling
5. High availability

1.4 Evolution of Cloud Data Warehouses

Early cloud warehouses moved traditional systems to cloud.

But problems remained:

- Compute and storage still coupled
- Performance bottlenecks
- Concurrency issues

This is where **Snowflake Inc.** changed the design completely.

1.5 Introduction to Snowflake

Snowflake was built from scratch for the cloud.

It is:

- Cloud-native
- Fully managed
- Multi-cloud
- Multi-cluster

Supported clouds:

- AWS
- Azure
- GCP

1.5.1 Core Philosophy

Separate Storage and Compute completely.

This single principle solves:

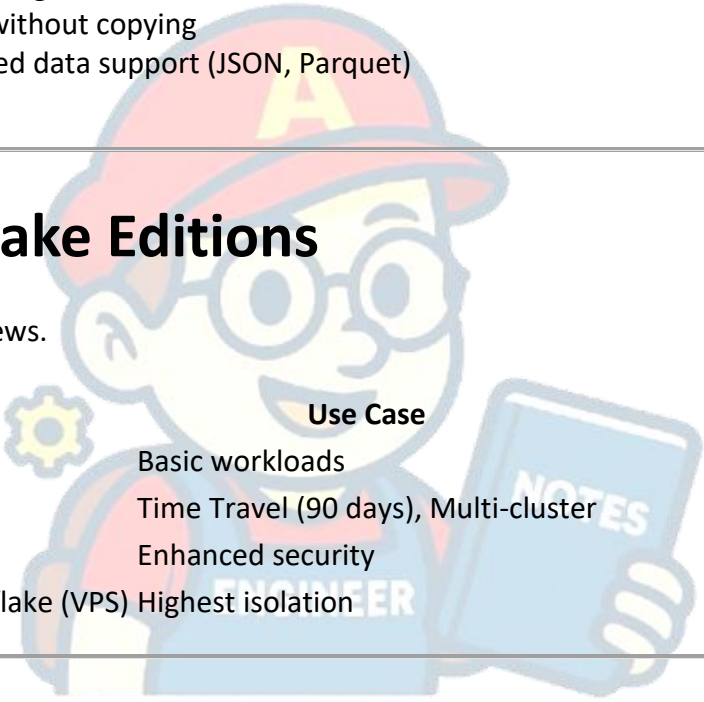
- Performance bottlenecks
 - Concurrency problems
 - Independent scaling
-

1.5.2 Why Companies Choose Snowflake

1. No infrastructure management
 2. Automatic performance optimization
 3. Strong security
 4. Zero-copy cloning
 5. Data sharing without copying
 6. Semi-structured data support (JSON, Parquet)
-

1.6 Snowflake Editions

Important for interviews.



Edition	Use Case
Standard	Basic workloads
Enterprise	Time Travel (90 days), Multi-cluster
Business Critical	Enhanced security
Virtual Private Snowflake (VPS)	Highest isolation

1.6.1 Time Travel Retention

- Standard → 1 day
 - Enterprise → up to 90 days
-

1.7 Pricing Model

Snowflake pricing is based on:

1. Storage (per TB per month)
2. Compute (per second usage of warehouse)
3. Cloud services (minimal cost)

Important Concept: Credits

Compute consumption is measured in **credits**.

Warehouse Size vs Credit Usage:

Warehouse Size Credits per Hour

X-Small	1
Small	2
Medium	4
Large	8

If warehouse runs 30 minutes:

- Small warehouse → 1 credit charged

1.8 Real-World Scenario

Company has:

- BI Team
- ETL Team
- Data Science Team

In traditional systems:

- One system → heavy query blocks others

In Snowflake:

- Create separate virtual warehouses
- Each team works independently
- No performance impact

This is called **Workload Isolation**.

1.9 Training Account Creation

Steps:

1. Go to Snowflake website
2. Start 30-day trial
3. Choose cloud provider

4. Select region
 5. Create account
 6. Login using Web UI (Snowsight)
-

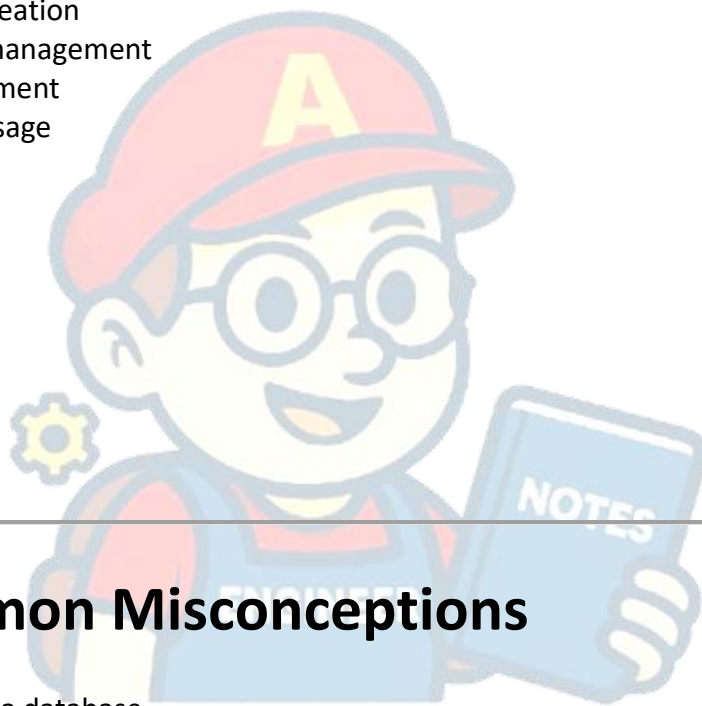
1.10 Web UI (Snowsight) Walkthrough

Snowsight allows:

- Query writing
- Worksheet creation
- Warehouse management
- Role management
- Monitoring usage

Key Sections:

- Databases
- Warehouses
- Admin
- Data
- Monitoring



1.11 Common Misconceptions

- ✗ Snowflake is just a database
 - ✓ It is a cloud data platform
 - ✗ Snowflake needs indexing
 - ✓ No indexing needed (automatic micro-partitions)
 - ✗ Scaling requires downtime
 - ✓ Scaling is instant
-

1.12 Summary of Chapter

In this chapter you learned:

- Why traditional warehouses failed to scale
- Cloud fundamentals

- Snowflake's origin and purpose
- Editions and pricing model
- Compute credit concept
- Workload isolation advantage

This foundation is crucial before understanding architecture.



Practice Exercises

Section A – Theory

1. Explain difference between OLTP and OLAP.
 2. Why was compute-storage coupling a problem?
 3. Why is Snowflake called SaaS?
 4. What is workload isolation?
 5. Explain credit consumption with example.
-

Section B – Practical

1. Create trial account.
 2. Create one warehouse (X-Small).
 3. Suspend and resume warehouse manually.
 4. Observe credit usage in monitoring tab.
-

Section C – Interview Preparation

1. Why did Snowflake succeed over traditional warehouses?
2. Explain Snowflake pricing model.
3. How would you reduce compute cost?
4. What happens if warehouse is not suspended?

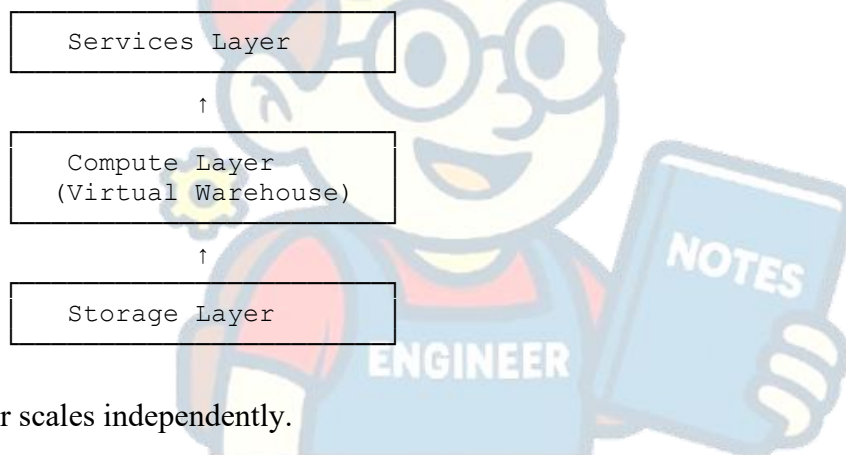
CHAPTER 2

SNOWFLAKE ARCHITECTURE – INTERNAL WORKING & PERFORMANCE DESIGN

2.1 Snowflake 3-Layer Architecture

Snowflake was built by Snowflake Inc. as a cloud-native system.

Its architecture has **three independent layers**:



Each layer scales independently.

This separation is Snowflake's greatest strength.

2.2 Storage Layer (Where Data Lives)

2.2.1 Where is Data Stored?

Snowflake stores data in cloud object storage:

- AWS → Amazon S3
- Azure → Blob Storage
- GCP → Cloud Storage

Data is:

- Automatically compressed

- Encrypted
- Columnar format

You do not manage storage manually.

2.2.2 Micro-Partitions (Very Important)

Snowflake automatically divides table data into **micro-partitions**.

Characteristics:

- ~16 MB compressed size
- Columnar storage
- Immutable
- Automatically created

You cannot manually create them.

2.2.3 Why Micro-Partitions Matter?

Snowflake maintains metadata for each micro-partition:

- Min value of column
- Max value
- Distinct count
- Null count

This allows **Partition Pruning**.

Example

Suppose sales table has:

```
SELECT * FROM sales
WHERE order_date = '2025-01-01';
```

If only 3 micro-partitions contain that date,
Snowflake scans only those 3 partitions.

No index needed.

2.2.4 Traditional Index vs Micro-Partition

Traditional DB:

- You create index manually
- Extra storage cost

Snowflake:

- Automatic metadata pruning
- No index maintenance
- No tuning required

This simplifies engineering.

2.3 Data Clustering

2.3.1 Natural Clustering

When you insert data in ordered manner, Snowflake automatically clusters data naturally.

Example:

If you load data sorted by `order_date`, micro-partitions will naturally align.

2.3.2 When Clustering is Needed?

If data is:

- Frequently filtered on specific column
- Very large table (TB+)
- High query latency

Then use clustering key.

Create Clustering Key

```
ALTER TABLE sales
CLUSTER BY (order_date);
```

Snowflake automatically reclusters in background.

2.3.3 Automatic Reclustering

Snowflake reorganizes partitions when needed.

But:

- Reclustering consumes compute credits.

So use only when necessary.

2.4 Compute Layer (Virtual Warehouses)

This is where queries run.

2.4.1 What is Virtual Warehouse?

A virtual warehouse is:

- Independent compute cluster
 - Can scale up (size)
 - Can scale out (multi-cluster)
 - Fully isolated
-

2.4.2 Warehouse Sizes

Size	Relative Power
X-Small	1x
Small	2x
Medium	4x
Large	8x

Each size doubles compute power.

2.4.3 Create Warehouse Example

```
CREATE WAREHOUSE etl_wh  
WAREHOUSE_SIZE = 'MEDIUM'
```

```
AUTO_SUSPEND = 60  
AUTO_RESUME = TRUE;
```

Important Parameters:

- AUTO_SUSPEND → reduces cost
 - AUTO_RESUME → starts automatically
 - MAX_CLUSTER_COUNT → for concurrency
-

2.4.4 Multi-Cluster Warehouse

Used for high concurrency.

Example:

```
CREATE WAREHOUSE bi_wh  
WAREHOUSE_SIZE = 'SMALL'  
MIN_CLUSTER_COUNT = 1  
MAX_CLUSTER_COUNT = 3  
SCALING_POLICY = 'STANDARD';
```

If 50 users run queries,
Snowflake automatically adds clusters.

2.4.5 Workload Isolation (Critical Concept)

Create separate warehouses for:

- ETL
- Reporting
- Data Science

Benefits:

- No query blocking
 - Predictable performance
 - Cost tracking per team
-

2.5 Services Layer (The Brain)

This layer handles:

- Authentication
- Authorization

- Query parsing
- Query optimization
- Metadata management
- Transaction control

You cannot see this layer,
but it is always active.

2.6 Query Lifecycle (Interview Favorite)

When you run a query:

- Step 1: Services layer parses SQL
- Step 2: Checks privileges
- Step 3: Optimizer creates execution plan
- Step 4: Compute warehouse executes
- Step 5: Data read from storage
- Step 6: Result cached

This entire flow happens in seconds.

2.7 Caching (Brief Introduction – Deep in Later Chapter)

Snowflake uses:

1. Result Cache
2. Metadata Cache
3. SSD Cache

First query slow → next query fast.

2.8 Real-World Architecture Design Example

Company has:

- 5 ETL pipelines
- 200 BI users
- Data Science experiments

Recommended design:

Team Warehouse		Size
ETL	etl_wh	Medium
BI	bi_wh	Small Multi-cluster
DS	ds_wh	Large (temporary)

This ensures:

- Cost control
- Performance stability
- Concurrency scaling

2.9 Common Mistakes

- ✗ Using one warehouse for everything
- ✗ Oversizing warehouse instead of optimizing query
- ✗ Ignoring auto-suspend
- ✗ Adding clustering unnecessarily
- ✗ Not monitoring credit usage

2.10 Interview Questions

1. Explain Snowflake 3-layer architecture.
2. What are micro-partitions?
3. How does partition pruning work?
4. Difference between scaling up and scaling out?
5. What is multi-cluster warehouse?
6. When would you use clustering key?
7. Explain query execution flow.



Practice Exercises

Basic

1. Create three warehouses (ETL, BI, DS).
2. Set auto-suspend to 60 seconds.
3. Resize warehouse from SMALL to MEDIUM.

Intermediate

4. Create large table and check query profile.
 5. Apply clustering key and observe difference.
 6. Create multi-cluster warehouse.
-

Advanced

7. Design architecture for:
 - 500 concurrent BI users
 - 10 ETL jobs
 - Strict cost control
 8. Explain how micro-partition metadata improves performance.
-



Chapter Summary

You now understand:

- Separation of storage & compute
- Micro-partitions and pruning
- Virtual warehouse scaling
- Multi-cluster concurrency
- Services layer responsibilities
- Query lifecycle

This chapter builds your architectural thinking.

Without this understanding,
you cannot optimize Snowflake.

CHAPTER 3

MICRO-PARTITIONS, CLUSTERING & CACHING

(Performance Engineering in Snowflake)

3.1 Micro-Partitions – Internal Structure

You already know:

- Each table is divided into ~16 MB compressed micro-partitions.
- They are automatically created.
- They are immutable.

Now let us understand **why immutability matters**.

3.1.1 Immutable Design

When you UPDATE or DELETE rows:

Snowflake does NOT modify existing micro-partition.

Instead:

- Creates new micro-partition
- Marks old one as logically deleted

This design enables:

- Time Travel
- Zero-copy cloning
- High concurrency
- No locking issues

Traditional systems rewrite blocks. Snowflake replaces them.

3.1.2 Columnar Storage

Snowflake stores data in **columnar format**.

Meaning:

If table has 10 columns,
and query uses only 2 columns,
only those 2 columns are scanned.

Example:

```
SELECT order_id, amount
FROM sales
WHERE order_date = '2025-01-01';
```

Unused columns are not read.

This reduces I/O significantly.

3.2 Partition Pruning – The Hidden Performance Engine

Snowflake stores metadata for each micro-partition:

- MIN(order_date)
- MAX(order_date)
- NULL count
- Distinct values

3.2.1 How Pruning Works

Suppose micro-partition metadata:

Partition	Min Date	Max Date
P1	2025-01-01	2025-01-10
P2	2025-02-01	2025-02-10

Query:

```
SELECT * FROM sales
WHERE order_date = '2025-02-05';
```

Snowflake scans only P2.

P1 is skipped completely.

This is called **Partition Pruning**.

3.2.2 Why Sometimes Pruning Fails

Pruning works best when:

- ✓ Direct column comparison
- ✓ Simple predicates

Pruning fails when:

- ✗ Function wrapping column

Example:

```
WHERE YEAR(order_date) = 2025
```

Better:

```
WHERE order_date BETWEEN '2025-01-01' AND '2025-12-31'
```

Engineer mindset:

Always write pruning-friendly filters.

3.3 Clustering – When and Why?

Micro-partitions are created based on insertion order.

If data is loaded randomly,
pruning efficiency reduces.

3.3.1 Natural Clustering

If ETL loads data sorted by order_date,
partitions naturally align.

No clustering key required.

3.3.2 Explicit Clustering Key

Used when:

- Table size is multiple TB
- Frequent filters on specific column
- Query performance degrading

Example:

```
ALTER TABLE sales  
CLUSTER BY (order_date, region);
```

Snowflake automatically reclusters.

3.3.3 Clustering Depth

Snowflake provides clustering depth metric.

Low depth → well clustered

High depth → poor clustering

Reclustering consumes compute credits.

So apply clustering only when required.

3.4 Snowflake Caching (Three Layers)

This is extremely important for interviews.

3.4.1 Result Cache

Stored in Services Layer.

If:

- Query text same
- Underlying data unchanged
- Same role/privileges

Then result returned instantly.

Valid for 24 hours.

Example:

```
SELECT COUNT(*) FROM sales;
```

Run again → instant result.

3.4.2 Metadata Cache

Stores:

- Micro-partition metadata
- Table statistics

Helps pruning faster.

3.4.3 SSD Cache (Warehouse Cache)

Stored inside Virtual Warehouse.

If warehouse is active,
data remains cached in SSD.

If warehouse suspended → cache cleared.

3.5 Query Profile – Performance Diagnosis Tool

Every Snowflake Engineer must know this.

In Snowsight:

1. Run query
2. Click Query Profile
3. Analyze:
 - Bytes scanned
 - Partitions scanned
 - Time spent
 - Spill to disk

3.5.1 What to Observe

If:

- 100% partitions scanned → poor pruning
 - High spill → warehouse too small
 - High scan time → clustering issue
-

3.6 Real-World Optimization Scenario

Scenario 1: Slow Report

Problem:

Dashboard query takes 90 seconds.

Investigation:

- 80% partitions scanned
- No clustering
- Filter on order_date

Solution:

- Add clustering key
- Or load data sorted
- Resize warehouse if needed

Scenario 2: High Credit Consumption

Problem:

Credits increasing rapidly.

Cause:

- Warehouse never suspends
- Large warehouse for small queries

Solution:

- Enable AUTO_SUSPEND
- Reduce warehouse size
- Separate ETL and BI workloads

3.7 Performance Optimization Strategy

Follow this order:

- 1 Check Query Profile
- 2 Verify pruning
- 3 Optimize SQL
- 4 Consider clustering
- 5 Resize warehouse (last option)

Do not increase warehouse blindly.

3.8 Common Performance Mistakes

- ✗ Using SELECT *
 - ✗ Applying functions on filter columns
 - ✗ Unnecessary clustering
 - ✗ Ignoring warehouse auto-suspend
 - ✗ Running heavy ETL in BI warehouse
-

3.9 Interview Questions

1. What is micro-partition immutability?
 2. Explain partition pruning.
 3. Difference between result cache and SSD cache?
 4. When does result cache not work?
 5. When should clustering be used?
 6. What happens when warehouse suspends?
 7. Why does second query run faster?
-



Practice Exercises

Section A – Practical

1. Create large sample table.
2. Run filtered query.

3. Check partitions scanned in Query Profile.
4. Rewrite query to improve pruning.
5. Suspend warehouse and rerun query.

Observe difference.

Section B – Intermediate

6. Load data randomly.
 7. Apply clustering key.
 8. Compare performance before and after.
-

Section C – Advanced

9. Design clustering strategy for:
 - 5 TB sales table
 - Frequent filtering by region & date
 10. Explain full query execution including caching and pruning.
-



Chapter Summary

You have now mastered:

- Micro-partition internals
- Immutable storage design
- Columnar scanning
- Partition pruning mechanics
- Natural vs explicit clustering
- Three caching layers
- Query Profile analysis

This knowledge makes you capable of:

- ✓ Diagnosing slow queries
- ✓ Reducing credit usage
- ✓ Designing high-performance architecture

CHAPTER 4

SECURITY & ACCESS CONTROL IN SNOWFLAKE

(Enterprise Governance Foundation)

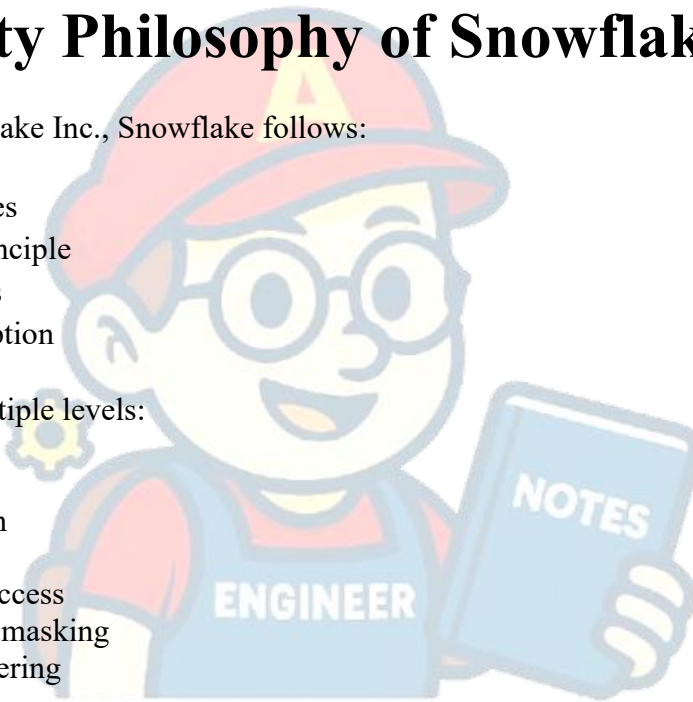
4.1 Security Philosophy of Snowflake

Developed by Snowflake Inc., Snowflake follows:

- ✓ Separation of Duties
- ✓ Least Privilege Principle
- ✓ Role-Based Access
- ✓ End-to-End Encryption

Security exists at multiple levels:

- Network
- Authentication
- Authorization
- Object-level access
- Column-level masking
- Row-level filtering



4.2 Access Control Models

Snowflake supports:

- 1** RBAC – Role-Based Access Control
 - 2** DAC – Discretionary Access Control
-

4.3 RBAC (Role-Based Access Control)

This is the primary model used.

Instead of giving permissions directly to users,
we assign permissions to roles.

Then assign roles to users.

4.3.1 Why RBAC?

Imagine company has:

- 50 Analysts
- 10 Data Engineers
- 5 Admins

If you assign privileges directly to users,
management becomes impossible.

Instead:

Create roles:

- ANALYST_ROLE
- ETL_ROLE
- ADMIN_ROLE

Grant privileges to roles.
Assign roles to users.

4.3.2 Core Security Objects

Object	Purpose
USER	Login identity
ROLE	Collection of privileges
PRIVILEGE	Permission (SELECT, INSERT, etc.)
WAREHOUSE	Compute access control
DATABASE	Data container

4.4 Creating Users & Roles

4.4.1 Create Role

```
CREATE ROLE analyst_role;
```

4.4.2 Grant Privileges to Role

```
GRANT SELECT ON TABLE sales TO ROLE analyst_role;  
GRANT USAGE ON WAREHOUSE reporting_wh TO ROLE analyst_role;  
GRANT USAGE ON DATABASE prod_db TO ROLE analyst_role;
```

4.4.3 Create User

```
CREATE USER abhi  
PASSWORD = 'Temp@123'  
DEFAULT_ROLE = analyst_role;
```

4.4.4 Assign Role to User

```
GRANT ROLE analyst_role TO USER abhi;
```

Now user inherits all privileges from role.

4.5 Hierarchical Roles (Role Inheritance)

Roles can inherit from other roles.

Example:

```
GRANT ROLE analyst_role TO ROLE senior_analyst_role;
```

Now senior analyst gets analyst permissions automatically.

This makes access scalable.

4.6 DAC (Discretionary Access Control)

In DAC:

Object owner can grant privileges.

Example:

If you create table:

```
CREATE TABLE test_table (...);
```

You become owner.

You can grant access:

```
GRANT SELECT ON test_table TO ROLE analyst_role;
```

4.7 Principle of Least Privilege

Always give:

- ✓ Only required privileges
- ✗ Never grant ACCOUNTADMIN unnecessarily

Best Practice:

Use custom roles instead of system roles.

4.8 Row Access Policy (Row-Level Security)

Used to restrict rows based on user role.

Example Scenario:

- Sales Manager should see only their region.
-

4.8.1 Create Row Access Policy

```
CREATE ROW ACCESS POLICY region_policy
AS (region VARCHAR) RETURNS BOOLEAN ->
CASE
  WHEN CURRENT_ROLE() = 'NORTH_ROLE' AND region = 'NORTH' THEN TRUE
  WHEN CURRENT_ROLE() = 'SOUTH_ROLE' AND region = 'SOUTH' THEN TRUE
  ELSE FALSE
END;
```

4.8.2 Apply Policy

```
ALTER TABLE sales
ADD ROW ACCESS POLICY region_policy ON (region);
```

Now data is filtered automatically.

4.9 Dynamic Data Masking (Column-Level Security)

Used to hide sensitive columns.

Example:

Salary column should be visible only to HR.

4.9.1 Create Masking Policy

```
CREATE MASKING POLICY mask_salary
AS (val NUMBER) RETURNS NUMBER ->
CASE
  WHEN CURRENT_ROLE() = 'HR_ROLE' THEN val
  ELSE NULL
END;
```

4.9.2 Apply Masking

```
ALTER TABLE employee
MODIFY COLUMN salary
SET MASKING POLICY mask_salary;
```

Now non-HR users see NULL instead of salary.

4.10 Tag-Based Masking

Instead of applying masking manually on each column, we use TAG.

4.10.1 Create Tag

```
CREATE TAG sensitive_data;
```

Assign tag:

```
ALTER TABLE employee
MODIFY COLUMN salary
SET TAG sensitive_data = 'yes';
```

Then apply masking policy on tag.

This ensures scalable governance.

4.11 Real-World Enterprise Security Design

Company Structure:

- ETL Team
- BI Team
- HR Team
- Finance Team

Recommended Design:

Role	Access
ETL_ROLE	INSERT/UPDATE
BI_ROLE	SELECT
HR_ROLE	SELECT + Salary View
FINANCE_ROLE	Financial Tables

Separate warehouse access per team.

Apply:

- Row Access Policy for region
- Masking for salary
- Tag-based governance for PII

4.12 Common Security Mistakes

- ✗ Granting privileges directly to users
 - ✗ Using ACCOUNTADMIN for daily tasks
 - ✗ Forgetting warehouse USAGE privilege
 - ✗ Ignoring row-level policies
 - ✗ Not auditing role hierarchy
-

4.13 Interview Questions

1. Explain RBAC in Snowflake.
2. Difference between RBAC and DAC?
3. How do you implement row-level security?

4. How does dynamic data masking work?
 5. What is tag-based masking?
 6. Why follow least privilege principle?
 7. How would you design access for multi-team organization?
-



Practice Exercises

Basic

1. Create role for Analyst.
 2. Grant SELECT privilege.
 3. Assign role to user.
-

Intermediate

4. Create row access policy for region-based access.
 5. Create masking policy for salary column.
 6. Test with different roles.
-

Advanced

7. Design full role hierarchy for:
 - o 100 analysts
 - o 20 ETL engineers
 - o 5 admins
 8. Implement least privilege strategy.
 9. Explain how you would audit access history.
-



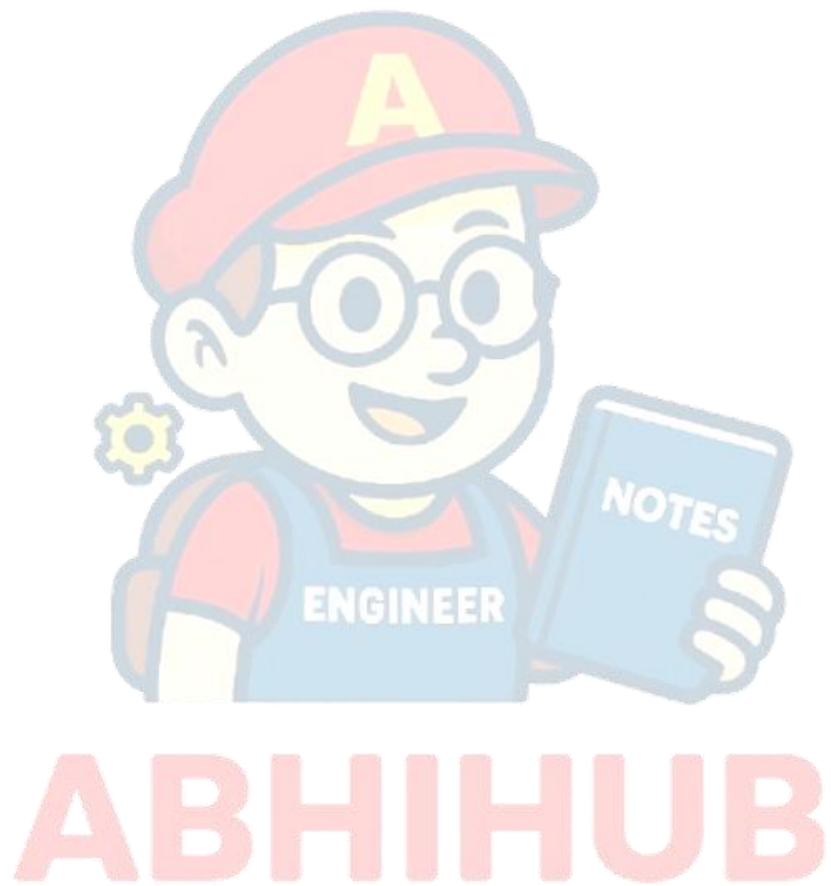
Chapter Summary

You now understand:

- ✓ Role-Based Access Control
- ✓ Role hierarchy
- ✓ Row-level security
- ✓ Column-level masking
- ✓ Tag-based governance
- ✓ Enterprise security design

Security in Snowflake is powerful because:

- It is centralized
- It is policy-driven
- It is scalable



CHAPTER 5

CONTINUOUS DATA PROTECTION

(Time Travel, Fail-Safe & Cloning)

5.1 Why Continuous Data Protection?

Traditional databases required:

- Manual backups
- Restore scripts
- Downtime
- DBA intervention

Snowflake eliminates traditional backup complexity.

Built by Snowflake Inc. with immutable micro-partitions, Snowflake makes recovery almost effortless.

Because data is immutable, old versions remain available for some time.

5.2 Time Travel

5.2.1 What is Time Travel?

Time Travel allows you to:

- Query historical data
- Restore dropped objects
- Undo accidental changes

Without restoring from backup.

5.2.2 Retention Period

Depends on edition:

Edition	Retention
Standard	1 day
Enterprise	Up to 90 days

Retention can be configured at:

- Database level
- Schema level
- Table level

Example:

```
ALTER TABLE sales  
SET DATA_RETENTION_TIME_IN_DAYS = 7;
```

5.3 Querying Historical Data

5.3.1 Using Timestamp

```
SELECT * FROM sales  
AT (TIMESTAMP => '2026-02-15 10:00:00');
```

5.3.2 Using Offset (Seconds)

```
SELECT * FROM sales  
AT (OFFSET => -3600);
```

This shows data 1 hour ago.

5.4 Undrop Object

If table accidentally dropped:

```
UNDROP TABLE sales;
```

As long as retention period not expired.

This works for:

- Table
 - Schema
 - Database
-

5.5 Restoring Specific Data

Suppose wrong UPDATE executed:

```
UPDATE employee SET salary = 0;
```

You can restore:

```
CREATE TABLE employee_restore AS  
SELECT * FROM employee  
AT (OFFSET => -600);
```

Then replace original table.

This saves production.

5.6 Fail-safe

Time Travel is user-accessible.

Fail-safe is:

- Additional 7 days
- Only Snowflake Support can restore
- Not directly accessible to users

Purpose:

Disaster recovery.

Total protection window:

Retention Period + 7 days Fail-safe.

Example:

Enterprise (90 days) → 97 days protection.

5.7 Zero-Copy Cloning (Powerful Feature)

This is one of Snowflake's most impressive features.

5.7.1 What is Zero-Copy Clone?

Instant copy of:

- Table
- Schema
- Database

Without copying data physically.

Example:

```
CREATE TABLE sales_dev CLONE sales;
```

5.7.2 How It Works

Because micro-partitions are immutable:

- Clone references same storage
- No additional storage initially
- Storage increases only if changes happen

This is called **Copy-on-Write** mechanism.

5.8 Real-World Use Cases

5.8.1 Dev/Test Environment

Clone production database:

```
CREATE DATABASE dev_db CLONE prod_db;
```

Develop safely.

5.8.2 Data Experimentation

Data scientist can clone large dataset instantly, without waiting hours.

5.8.3 Backup Before Deployment

Before major update:

```
CREATE TABLE backup_sales CLONE sales;
```

If failure occurs → revert easily.

5.9 Storage Impact of Time Travel & Cloning

Important for cost understanding.

Because data is immutable:

- Updates create new micro-partitions
- Old partitions retained until retention expires

Heavy UPDATE workloads increase storage.

So:

- ✓ Use MERGE efficiently
 - ✓ Avoid unnecessary full-table updates
-

5.10 Comparing Time Travel vs Fail-safe

Feature	Time Travel	Fail-safe
User Access	Yes	No
Retention	1–90 days	7 days
Restore Type	Query & Undrop	Support Ticket
Purpose	Operational Recovery	Disaster Recovery

5.11 Enterprise Recovery Scenario

Scenario:

ETL pipeline corrupts 10 TB table.

Steps:

1. Identify corruption time.
2. Query historical version.
3. Create restore table.
4. Swap tables.

```
ALTER TABLE sales SWAP WITH sales_restore;
```

Zero downtime recovery.

5.12 Best Practices

- ✓ Set retention carefully (cost vs safety)
 - ✓ Clone before major production changes
 - ✓ Monitor storage growth
 - ✓ Avoid frequent large updates
 - ✓ Use SWAP for safe replacement
-

5.13 Common Mistakes

- ✗ Thinking Time Travel is permanent backup
 - ✗ Ignoring storage cost impact
 - ✗ Forgetting retention expiration
 - ✗ Not cloning before risky deployment
-

5.14 Interview Questions

1. Explain Time Travel in Snowflake.
 2. Difference between Time Travel and Fail-safe?
 3. How does zero-copy cloning work?
 4. What happens when retention period expires?
 5. Does cloning duplicate storage?
 6. How would you recover from accidental delete?
 7. How does immutable storage enable CDP?
-



Practice Exercises

Basic

1. Create table and insert records.
2. Delete some rows.
3. Query data using OFFSET.

Intermediate

4. Drop table and restore using UNDROP.
 5. Clone table and modify clone.
 6. Compare storage usage.
-

Advanced

7. Simulate production corruption and restore using SWAP.
 8. Design backup strategy for 50 TB warehouse.
 9. Calculate storage impact of frequent updates.
-



Chapter Summary

You now understand:

- ✓ Immutable micro-partitions
- ✓ Time Travel querying
- ✓ Undrop mechanism
- ✓ Fail-safe concept
- ✓ Zero-copy cloning
- ✓ Storage implications

This makes you capable of:

- Handling production incidents
- Designing safe deployment pipelines
- Protecting enterprise data

■ CHAPTER 6

DATA LOADING & BULK INGESTION

(CSV, JSON, Stage, COPY INTO, SnowSQL)

6.1 Types of Data Loading in Snowflake

There are two main methods:

1 Bulk Loading

Used for:

- Large historical files
- Batch ingestion
- Daily ETL

2 Continuous Loading

Used for:

- Near real-time data
- Streaming pipelines
(Streams & Snowpipe – covered later)

In this chapter, we focus on **Bulk Loading**.

6.2 What is a Stage?

A **Stage** is a location where data files are stored before loading into tables.

Think of it as a “temporary landing area”.

Snowflake supports:

- Internal Stage
 - External Stage
-

6.3 Internal Stage

Stored inside Snowflake's cloud storage.

Types:

- User Stage (@~)
- Table Stage (@%table_name)
- Named Stage (@stage_name)

6.3.1 Create Named Stage

```
CREATE STAGE my_stage;
```

6.3.2 Upload File to Stage

Using SnowSQL:

```
PUT file:///C:/data/sales.csv @my_stage;
```

File now available in Snowflake.

6.4 External Stage

Used when data already exists in cloud storage like:

- AWS S3
- Azure Blob
- GCS

Example:

```
CREATE STAGE ext_stage
URL='s3://my-bucket/data/'
CREDENTIALS=(AWS_KEY_ID='xxx' AWS_SECRET_KEY='yyy');
```

External stage avoids uploading files manually.

6.5 File Formats

Before loading data, Snowflake must understand file structure.

6.5.1 CSV File Format

```
CREATE FILE FORMAT csv_format
TYPE = 'CSV'
FIELD_DELIMITER = ','
SKIP_HEADER = 1
NULL_IF = ('NULL');
```

6.5.2 JSON File Format

```
CREATE FILE FORMAT json_format
TYPE = 'JSON';
```

Snowflake supports semi-structured data natively.

6.6 COPY INTO Command (Most Important)

This command loads data from stage to table.

6.6.1 Load CSV

```
COPY INTO sales
FROM @my_stage/sales.csv
FILE_FORMAT = csv_format;
```

6.6.2 Load JSON

```
COPY INTO raw_table
FROM @my_stage/data.json
FILE_FORMAT = json_format;
```

6.7 Handling Errors During Load

Important options:

```
COPY INTO sales
```

```
FROM @my_stage
FILE_FORMAT = csv_format
ON_ERROR = 'CONTINUE';
```

Options:

- ABORT_STATEMENT (default)
- CONTINUE
- SKIP_FILE

Best Practice:

Log rejected records for analysis.

6.8 Transform While Loading

You can transform data during load.

Example:

```
COPY INTO sales
FROM (
    SELECT $1, $2, TO_DATE($3)
    FROM @my_stage
)
FILE_FORMAT = csv_format;
```

Here:

- \$1, \$2 → file columns
- Apply transformation before insert

6.9 Loading JSON into VARIANT Column

Create table:

```
CREATE TABLE raw_data (
    data VARIANT
);
```

Load JSON:

```
COPY INTO raw_data
FROM @my_stage
FILE_FORMAT = json_format;
```

Query JSON:

```
SELECT data:name::STRING
FROM raw_data;
```

Snowflake stores JSON in VARIANT type.

6.10 SnowSQL (Command-Line Tool)

SnowSQL allows:

- Upload files
- Run queries
- Automate scripts

Developed by Snowflake Inc..

6.10.1 Install SnowSQL

Download from Snowflake website.

Configure:

```
snowsql -a <account> -u <username>
```

6.10.2 Automating Script Execution

```
snowsql -f script.sql
```

Used in ETL automation.

6.11 Unloading Data (Export)

Export table data to stage.

6.11.1 Unload to CSV

```
COPY INTO @my_stage/export_sales
FROM sales
FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',');
```

6.11.2 Unload JSON

```
COPY INTO @my_stage/export_json  
FROM raw_data  
FILE_FORMAT = (TYPE = JSON);
```

Used for:

- Data sharing
 - Backup
 - Downstream systems
-

6.12 Real-World Ingestion Architecture

Scenario:

Company receives:

- Daily CSV sales file
- Weekly JSON API data
- Monthly finance report

Design:

1. External stage connected to S3
2. Separate warehouse for ETL
3. COPY INTO scheduled via Task
4. Log load history
5. Validate row count

Always separate ingestion warehouse from BI warehouse.

6.13 Best Practices for Bulk Loading

- ✓ Use large files (100–250 MB compressed)
 - ✓ Compress files (GZIP)
 - ✓ Parallel loading
 - ✓ Use dedicated warehouse
 - ✓ Monitor load history
 - ✓ Avoid tiny files
-

6.14 Common Mistakes

- ✗ Loading small files repeatedly
 - ✗ Ignoring ON_ERROR behavior
 - ✗ Running load in BI warehouse
 - ✗ Not validating load results
 - ✗ Hardcoding credentials in SQL
-

6.15 Interview Questions

1. What is a stage in Snowflake?
 2. Difference between internal and external stage?
 3. Explain COPY INTO command.
 4. How do you handle errors during load?
 5. How does Snowflake store JSON?
 6. What is SnowSQL used for?
 7. How do you optimize bulk loading performance?
-



Practice Exercises

Basic

1. Create named stage.
 2. Upload CSV file.
 3. Load into table.
-

Intermediate

4. Create JSON table with VARIANT.
 5. Load JSON data.
 6. Query nested JSON fields.
-

Advanced

7. Design ingestion pipeline for 1 TB daily load.
8. Implement error handling strategy.
9. Compare internal vs external stage usage.

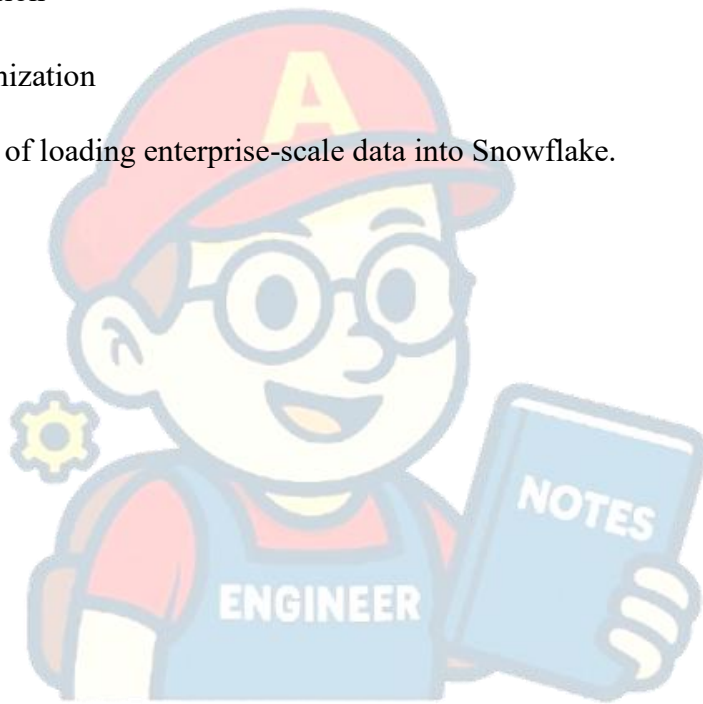


Chapter Summary

You now understand:

- ✓ Internal & external stages
- ✓ File formats
- ✓ COPY INTO
- ✓ JSON handling
- ✓ SnowSQL automation
- ✓ Data unloading
- ✓ Bulk loading optimization

You are now capable of loading enterprise-scale data into Snowflake.



ABHIHUB

■ CHAPTER 7

STREAMS, TASKS & CONTINUOUS DATA PIPELINE

(Incremental Processing & Automation)

7.1 Why Incremental Processing?

Imagine:

- Sales table has 5 TB data
- New data arrives every hour
- You reprocess full table every time

This wastes:

- Compute
- Time
- Money

Solution:

Process only changed data.

This is called **Change Data Capture (CDC)**.

7.2 What is a Stream?

A **Stream** tracks changes in a table.

It captures:

- INSERT
- UPDATE
- DELETE

Without modifying original table.

Think of stream as a “change log”.

7.2.1 How Stream Works Internally

Because Snowflake uses immutable micro-partitions:

- Every change creates new partition
- Stream reads difference between versions

No triggers required.

7.3 Creating a Stream

Example:

```
CREATE STREAM sales_stream  
ON TABLE sales;
```

Now:

If new records inserted into sales,
sales_stream will capture those changes.

7.4 Viewing Stream Data

```
SELECT * FROM sales_stream;
```

You will see metadata columns:

- METADATA\$ACTION → INSERT / DELETE
 - METADATA\$ISUPDATE → TRUE/FALSE
-

7.5 Using Stream for Incremental Load

Example Scenario:

Raw table → Target table

Step 1: Insert new records into raw table

```
INSERT INTO raw_sales VALUES (...);
```

Step 2: Process only changes

```
INSERT INTO final_sales  
SELECT * FROM raw_sales_stream;
```

Stream automatically clears consumed records.

This ensures incremental processing.

7.6 Types of Streams

1 Standard Stream

Tracks all changes

2 Append-Only Stream

Tracks only INSERT

Used when updates/deletes not needed.

7.7 What is a Task?

A **Task** automates SQL execution.

Instead of running manually,
Snowflake runs it on schedule.

7.8 Creating a Task

Example:

```
CREATE TASK process_sales_task  
WAREHOUSE = etl_wh  
SCHEDULE = '1 MINUTE'  
AS  
INSERT INTO final_sales  
SELECT * FROM sales_stream;
```

Start task:

```
ALTER TASK process_sales_task RESUME;
```

Now every minute:

- Stream data processed
 - Target table updated
-

7.9 Task Dependency (Pipeline Chaining)

You can chain tasks.

Example:

Task 1 → Clean data
Task 2 → Aggregate
Task 3 → Load report

```
CREATE TASK task2  
AFTER task1  
AS  
...
```

This builds full workflow.

7.10 Snowpipe (Continuous File Ingestion)

While Streams track table changes,
Snowpipe loads files automatically as soon as they arrive.

Snowpipe monitors external stage.

When file arrives:

- COPY INTO runs automatically.

Used for near real-time ingestion.

7.11 Building Complete CDC Pipeline

Real-world architecture:

1. External Stage (S3)

2. Snowpipe loads raw table
3. Stream tracks changes
4. Task processes incrementally
5. Target reporting table updated

This reduces:

- Full table scans
- Compute usage
- Latency

7.12 Example: End-to-End Pipeline

Step 1: Create Raw Table

```
CREATE TABLE raw_sales (...);
```

Step 2: Create Stream

```
CREATE STREAM raw_sales_stream  
ON TABLE raw_sales;
```

Step 3: Create Target Table

```
CREATE TABLE sales_final (...);
```

Step 4: Create Task

```
CREATE TASK load_sales_task  
WAREHOUSE = etl_wh  
SCHEDULE = '5 MINUTE'  
AS  
MERGE INTO sales_final t  
USING raw_sales_stream s  
ON t.id = s.id  
WHEN MATCHED THEN UPDATE SET ...  
WHEN NOT MATCHED THEN INSERT ...;
```

Step 5: Resume Task

```
ALTER TASK load_sales_task RESUME;
```

Now pipeline runs automatically.

7.13 Cost Control in Pipelines

- ✓ Use dedicated small warehouse
 - ✓ Set AUTO_SUSPEND
 - ✓ Avoid frequent tiny schedules
 - ✓ Monitor task history
-

7.14 Common Mistakes

- ✗ Forgetting to resume task
 - ✗ Consuming stream without storing data
 - ✗ Using large warehouse for small incremental load
 - ✗ Ignoring task failure logs
 - ✗ Running full refresh instead of MERGE
-

7.15 Interview Questions

1. What is a Stream in Snowflake?
 2. How does Snowflake implement CDC?
 3. Difference between Stream and Snowpipe?
 4. What happens after stream is consumed?
 5. How do you automate ETL in Snowflake?
 6. How would you design real-time pipeline?
 7. Explain task chaining.
-



Practice Exercises

Basic

1. Create table and stream.
 2. Insert records.
 3. Query stream output.
-

Intermediate

4. Create task to process stream.
 5. Implement MERGE logic.
 6. Check task history.
-

Advanced

7. Design real-time ingestion pipeline for:
 - 10 million rows per hour
 - 500 concurrent dashboard users
 8. Compare full refresh vs incremental approach.
 9. Explain how immutable storage enables stream functionality.
-

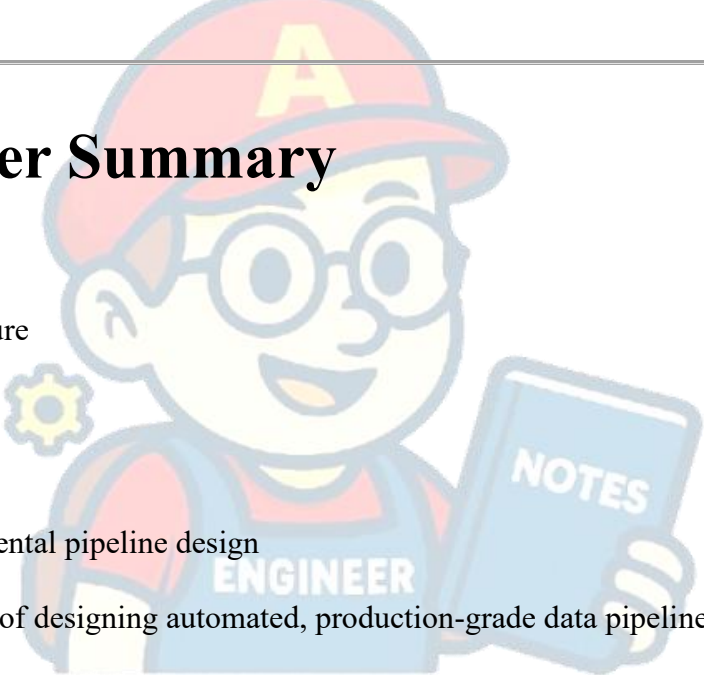


Chapter Summary

You now understand:

- ✓ Change Data Capture
- ✓ Streams
- ✓ Tasks
- ✓ Task chaining
- ✓ Snowpipe concept
- ✓ End-to-end incremental pipeline design

You are now capable of designing automated, production-grade data pipelines.

A cartoon illustration of a male engineer wearing a red hard hat with a yellow 'A' on it, blue overalls, and glasses. He is holding a blue book labeled 'NOTES' and has a yellow gear icon near his shoulder. The word 'ENGINEER' is written on his chest.

ABHIHUB

CHAPTER 8

PROGRAMMING & ADVANCED FEATURES IN SNOWFLAKE

8.1 User Defined Functions (UDF)

8.1.1 What is a UDF?

A UDF is a reusable function that:

- Takes input
- Returns output
- Cannot modify data

Think of it like a formula.

Used for:

- Data transformations
- Standardizing logic
- Reusable calculations

8.1.2 SQL UDF Example

```
CREATE FUNCTION calculate_tax(amount NUMBER)
RETURNS NUMBER
AS
$$
    amount * 0.18
$$;
```

Usage:

```
SELECT calculate_tax(1000);
```

8.1.3 JavaScript UDF

```
CREATE FUNCTION greet(name STRING)
RETURNS STRING
LANGUAGE JAVASCRIPT
AS
```

```
$$  
    return "Hello " + name;  
$$;
```

8.1.4 When to Use UDF?

- ✓ Standard calculations
 - ✓ Data cleansing logic
 - ✓ Repeated expressions
 - ✗ Do not use for heavy processing
-

8.2 Stored Procedures

8.2.1 What is a Stored Procedure?

Unlike UDF:

- Can execute multiple SQL statements
- Can modify data
- Can implement control flow

Supported languages:

- SQL (Snowflake Scripting)
- JavaScript

8.2.2 SQL Stored Procedure Example

```
CREATE PROCEDURE load_sales()  
RETURNS STRING  
LANGUAGE SQL  
AS  
$$  
BEGIN  
    INSERT INTO final_sales  
    SELECT * FROM raw_sales_stream;  
    RETURN 'Load Completed';  
END;  
$$;
```

Call procedure:

```
CALL load_sales();
```

8.2.3 JavaScript Procedure Example

```
CREATE PROCEDURE row_count()
RETURNS STRING
LANGUAGE JAVASCRIPT
AS
$$
var result = snowflake.execute(
  {sqlText: "SELECT COUNT(*) FROM sales"}
);
result.next();
return "Rows: " + result.getColumnValue(1);
$$;
```

8.3 UDF vs Stored Procedure

Feature	UDF Procedure	
Returns value	Yes	Yes
Modify data	No	Yes
Multiple statements	No	Yes
Used in SELECT	Yes	No

8.4 External Functions

Used when you need to call external API from Snowflake.

Example use cases:

- Call ML model API
- Currency conversion API
- Fraud detection system

External functions connect through cloud services.

Example Concept

```
CREATE EXTERNAL FUNCTION get_credit_score(...)
RETURNS NUMBER
API_INTEGRATION = my_api_integration
AS 'https://api.company.com/score';
```

Snowflake sends data → API → returns result.

8.5 External Tokenization

Used for:

- Masking sensitive data using external systems
- PCI compliance

Example:

Credit card numbers tokenized via external security service.

This ensures:

- Snowflake never stores raw sensitive value

8.6 Search Optimization Service

Used to improve performance of selective queries.

If table has:

- Large dataset
- Frequent point lookup

Example:

```
SELECT * FROM customers
WHERE email = 'abc@gmail.com';
```

Without clustering, this may scan many partitions.

Search Optimization builds additional metadata structure.

Enable:

```
ALTER TABLE customers
ADD SEARCH OPTIMIZATION;
```

Consumes extra storage & compute.

Use only when needed.

8.7 Dynamic Tables

Dynamic Tables are next-generation transformation feature.

They automatically maintain transformed data.

Instead of task + stream combination,
Snowflake manages incremental refresh automatically.

Example:

```
CREATE DYNAMIC TABLE sales_summary
TARGET_LAG = '5 MINUTE'
WAREHOUSE = etl_wh
AS
SELECT region, SUM(amount)
FROM sales
GROUP BY region;
```

Snowflake refreshes automatically.

Use case:

- Real-time dashboards
- Incremental aggregates

8.8 Hybrid Tables

Hybrid tables combine:

- OLTP + OLAP capabilities

They support:

- Primary key enforcement
- Low-latency transactions

Used for:

- Applications requiring transactional behavior inside Snowflake

8.9 Real-World Implementation Scenario

Company needs:

- Clean sales data
- Calculate tax
- Call external fraud API
- Maintain live summary dashboard

Design:

1. UDF → Tax calculation
2. Procedure → ETL orchestration
3. External function → Fraud scoring
4. Dynamic table → Summary refresh
5. Search optimization → Customer lookup

This shows how features combine.

8.10 Common Mistakes

- ✗ Using procedure when UDF enough
 - ✗ Writing heavy logic inside UDF
 - ✗ Ignoring cost of search optimization
 - ✗ Overusing dynamic tables without monitoring
-

8.11 Interview Questions

1. Difference between UDF and Procedure?
 2. When would you use JavaScript procedure?
 3. What is an external function?
 4. How does dynamic table differ from stream + task?
 5. When should search optimization be enabled?
 6. What is hybrid table?
 7. Explain real-world use case combining these features.
-



Practice Exercises

Basic

1. Create tax calculation UDF.
 2. Create stored procedure inserting records.
 3. Call procedure and verify result.
-

Intermediate

4. Build procedure to process stream data.

5. Create dynamic table for summary.
 6. Compare dynamic table vs task refresh.
-

Advanced

7. Design architecture for:
 - Fraud detection pipeline
 - Real-time aggregation
 - Secure tokenization
 8. Explain how external function integrates securely.
 9. Optimize lookup-heavy table using search optimization.
-



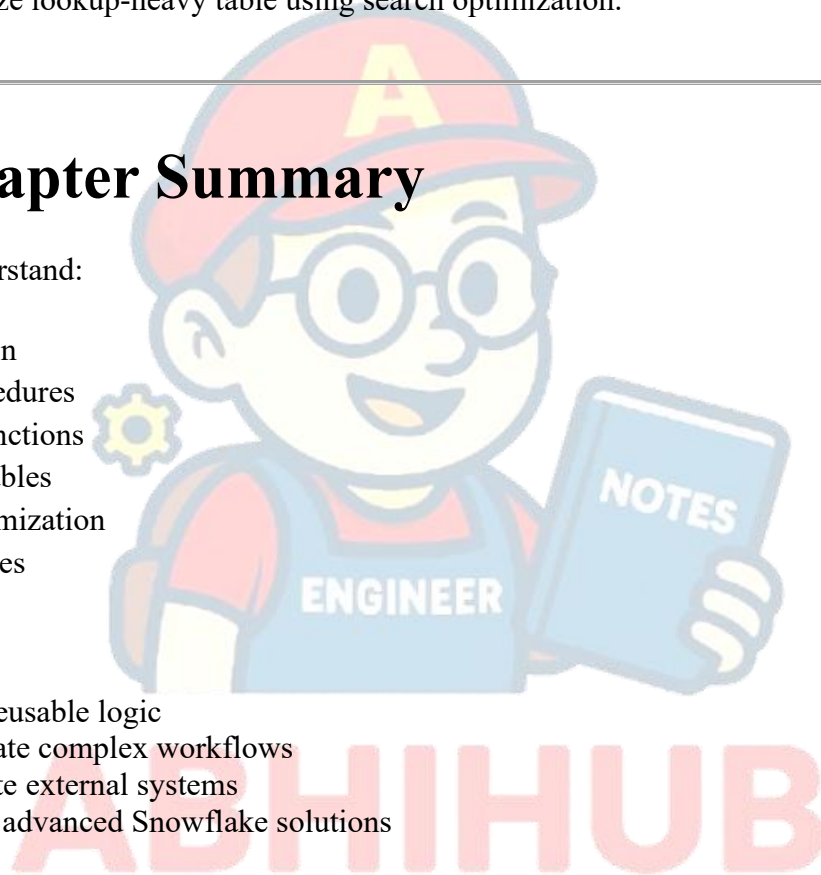
Chapter Summary

You now understand:

- ✓ UDF creation
- ✓ Stored Procedures
- ✓ External Functions
- ✓ Dynamic Tables
- ✓ Search Optimization
- ✓ Hybrid Tables

You can now:

- Build reusable logic
- Automate complex workflows
- Integrate external systems
- Design advanced Snowflake solutions



CHAPTER 9

MONITORING, RESOURCE MANAGEMENT & GOVERNANCE

9.1 Why Monitoring is Important?

In Snowflake:

- Compute costs money (credits)
- Storage costs money
- Cloud services cost money

Without monitoring:

- Budgets explode
- Queries run uncontrolled
- Teams misuse resources

A true engineer designs with visibility.

9.2 Resource Monitor

9.2.1 What is Resource Monitor?

A Resource Monitor tracks credit usage and enforces limits.

You can:

- Send alerts
 - Suspend warehouse
 - Suspend account
-

9.2.2 Create Resource Monitor

```
CREATE RESOURCE MONITOR monthly_limit
WITH CREDIT_QUOTA = 500
FREQUENCY = MONTHLY
START_TIMESTAMP = IMMEDIATELY
```

```
TRIGGERS
ON 80 PERCENT DO NOTIFY
ON 100 PERCENT DO SUSPEND;
```

This:

- Alerts at 80%
 - Suspends at 100%
-

9.2.3 Assign to Warehouse

```
ALTER WAREHOUSE etl_wh
SET RESOURCE_MONITOR = monthly_limit;
```

Now warehouse cannot exceed quota.

9.3 Monitoring Warehouse Usage

Use:

- Snowsight → Admin → Usage
- ACCOUNT_USAGE schema

Example:

```
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY;
```

This shows:

- Credits used
 - Warehouse name
 - Time range
-

9.4 Query Monitoring

Check slow queries:

```
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
ORDER BY START_TIME DESC;
```

Important columns:

- EXECUTION_TIME
- BYTES_SCANNED
- WAREHOUSE_SIZE
- USER_NAME

Use this for optimization.

9.5 Tagging (Data Governance Foundation)

Tags help classify objects.

Example use cases:

- PII data
 - Finance data
 - Confidential
 - Department ownership
-

9.5.1 Create Tag

```
CREATE TAG data_sensitivity;
```

9.5.2 Assign Tag

```
ALTER TABLE employee  
SET TAG data_sensitivity = 'CONFIDENTIAL';
```

Now table is categorized.

9.6 Tag-Based Masking Policies

Instead of applying masking manually,
apply policy to tag.

Example:

1. Create masking policy.
2. Attach masking policy to tag.

Now any column with that tag automatically masked.

This ensures scalable governance.

9.7 Data Classification

Snowflake provides automatic classification to detect:

- Email
- Phone
- SSN
- Credit card

This helps identify sensitive columns.

After classification:

- Apply masking
- Apply row access
- Track compliance

Important for regulatory environments.

9.8 Access History

Used to audit:

- Who accessed which table?
- Which role was used?
- When was it accessed?

Example:

```
SELECT *  
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY  
ORDER BY QUERY_START_TIME DESC;
```

Critical for:

- Security audits
 - Compliance (GDPR, HIPAA)
 - Incident investigation
-

9.9 Object Dependencies

Before dropping table,
you must know:

- Which views depend on it?
- Which procedures use it?

Example:

```
SELECT *  
FROM SNOWFLAKE.ACCOUNT_USAGE.OBJECT_DEPENDENCIES;
```

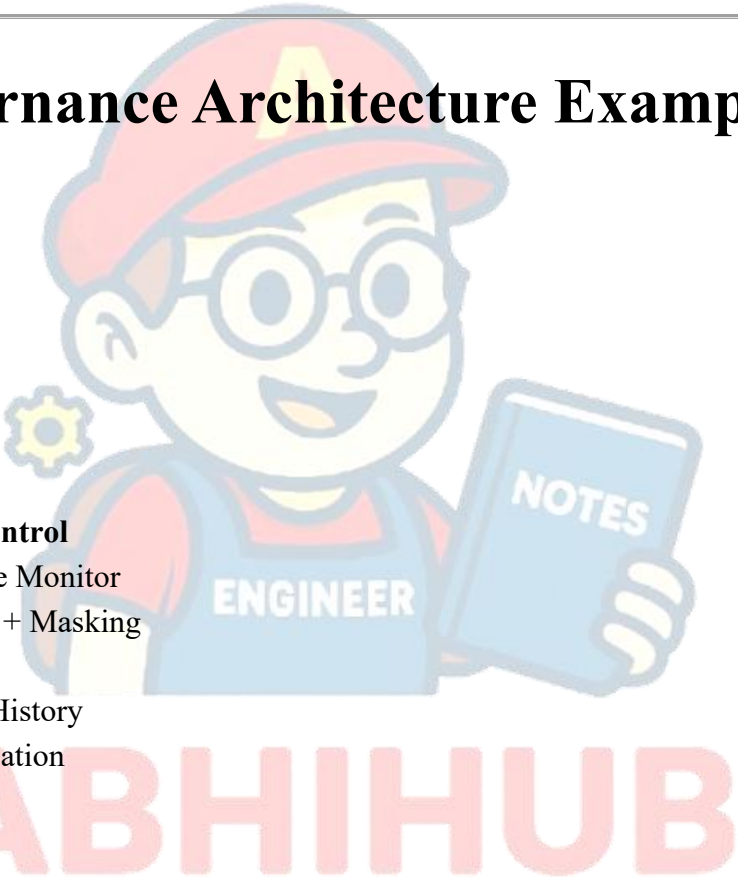
Prevents production failures.

9.10 Governance Architecture Example

Company Structure:

- HR
- Finance
- Sales
- Data Science

Governance Strategy:



Layer	Control
Compute	Resource Monitor
Data	Tagging + Masking
Access	RBAC
Audit	Access History
Compliance	Classification

This ensures:

- Cost control
- Data protection
- Visibility
- Accountability

9.11 Cost Optimization Strategy

- ✓ Auto-suspend warehouses
- ✓ Right-size warehouses
- ✓ Separate ETL & BI

- ✓ Avoid unnecessary clustering
 - ✓ Monitor query history weekly
 - ✓ Use multi-cluster only when needed
-

9.12 Common Governance Mistakes

- ✗ No resource monitor
 - ✗ Using large warehouse for small queries
 - ✗ No tagging strategy
 - ✗ Ignoring audit logs
 - ✗ Dropping objects without dependency check
-

9.13 Interview Questions

1. What is Resource Monitor?
 2. How do you track warehouse usage?
 3. How do you audit user access?
 4. What is tag-based governance?
 5. How do you prevent credit overuse?
 6. How do you find object dependencies?
 7. Design governance strategy for enterprise.
-



Practice Exercises

Basic

1. Create resource monitor.
 2. Assign to warehouse.
 3. Check warehouse usage.
-

Intermediate

4. Create tag and assign to table.
 5. Apply masking policy via tag.
 6. Query access history.
-

Advanced

7. Design cost-control strategy for:
 - 200 BI users
 - 20 ETL pipelines
 8. Build governance framework for financial company.
 9. Analyze query history and identify top cost queries.
-

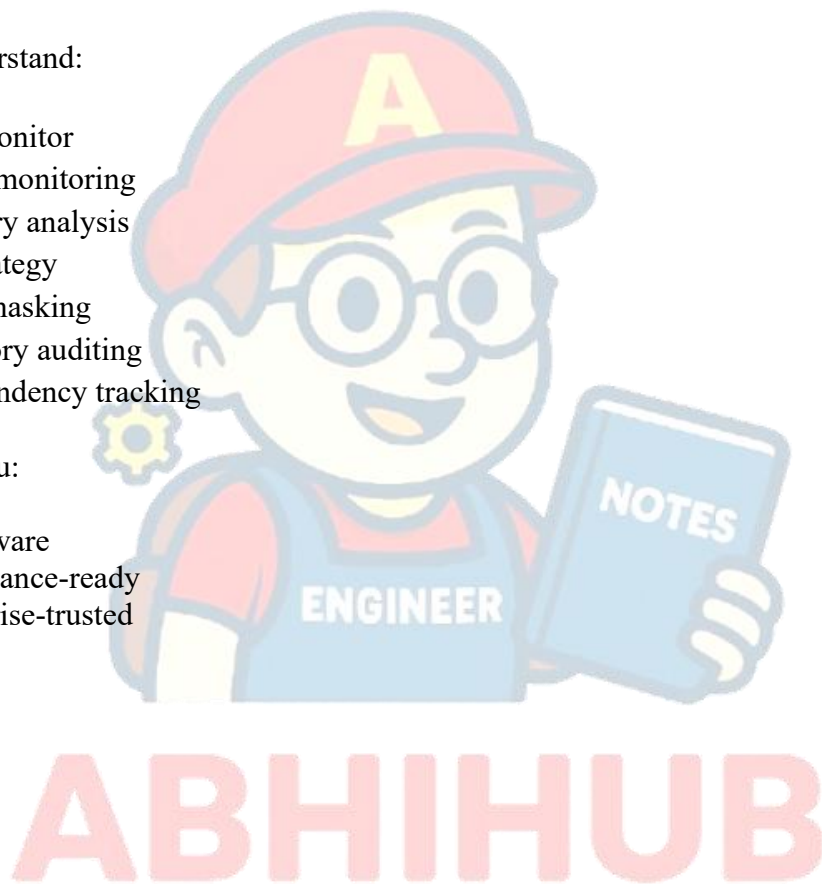
Chapter Summary

You now understand:

- ✓ Resource Monitor
- ✓ Warehouse monitoring
- ✓ Query history analysis
- ✓ Tagging strategy
- ✓ Tag-based masking
- ✓ Access history auditing
- ✓ Object dependency tracking

This makes you:

- Cost-aware
- Governance-ready
- Enterprise-trusted



CHAPTER 10

DATA SHARING, REPLICATION & CAPSTONE PROJECT

10.1 Why Data Sharing is Revolutionary?

Traditional method:

- Export data
- Transfer file
- Import into another system
- Duplicate storage

Problems:

- Slow
- Expensive
- Data inconsistency

Built by Snowflake Inc., Snowflake enables **Secure Data Sharing without copying data**.

10.2 Snowflake Secure Data Sharing

Data sharing allows:

- One account (Provider)
- Share data to another account (Consumer)
- Without duplicating storage

Consumer queries data directly.

10.3 Provider Steps

Step 1: Create Share

```
CREATE SHARE sales_share;
```

Step 2: Grant Access to Objects

```
GRANT USAGE ON DATABASE prod_db TO SHARE sales_share;  
GRANT USAGE ON SCHEMA prod_db.public TO SHARE sales_share;  
GRANT SELECT ON TABLE prod_db.public.sales TO SHARE sales_share;
```

Step 3: Add Consumer Account

```
ALTER SHARE sales_share  
ADD ACCOUNT = consumer_account_id;
```

10.4 Consumer Steps

Consumer creates database from share:

```
CREATE DATABASE shared_sales_db  
FROM SHARE provider_account.sales_share;
```

Now consumer can query:

```
SELECT * FROM shared_sales_db.public.sales;
```

Data remains in provider account.

10.5 Benefits of Secure Data Sharing

- ✓ No data copy
- ✓ No data movement
- ✓ Real-time updates
- ✓ Zero storage duplication
- ✓ Controlled access

Used by:

- Data providers
 - SaaS companies
 - Financial institutions
 - Data marketplaces
-

10.6 Database Replication

Used for:

- Disaster Recovery
- Cross-region availability
- Business continuity

Replication copies metadata and data to secondary region.

10.6.1 Enable Replication

```
ALTER DATABASE prod_db  
ENABLE REPLICATION TO ACCOUNTS target_account;
```

10.6.2 Create Secondary Database

```
CREATE DATABASE prod_db_secondary  
AS REPLICA OF source_account.prod_db;
```

10.6.3 Refresh Replica

```
ALTER DATABASE prod_db_secondary REFRESH;
```

10.7 Cross-Region & Cross-Cloud Replication

Supported across:

- AWS regions
- Azure regions
- GCP regions

Enterprise feature.

Used for:

- High availability
 - Regulatory compliance
 - Geo-distributed analytics
-

10.8 Failover & Business Continuity

If primary region fails:

- Promote secondary database
- Resume operations

Critical for financial systems.

10.9 Advanced Unloading Patterns

Used for:

- Sharing with non-Snowflake systems
 - Data lake export
 - Backup archive
-

10.9.1 Unload to External Stage

```
COPY INTO @external_stage/sales_export
FROM sales
FILE_FORMAT = (TYPE = CSV);
```

10.9.2 Partitioned Unload

```
COPY INTO @external_stage/sales_export
FROM (
    SELECT * FROM sales
)
FILE_FORMAT = (TYPE = CSV)
PARTITION BY region;
```

Useful for large-scale distributed processing.

10.10 Capstone Project – End-to-End Snowflake Implementation

Now we design like enterprise engineer.

Business Scenario

Company Requirements:

- Daily CSV sales file
 - Real-time JSON API data
 - 300 BI users
 - Strict data governance
 - Disaster recovery requirement
 - Cost control policy
-

10.11 Step-by-Step Architecture Design

1 Storage & Ingestion

- External Stage (S3)
 - Snowpipe for real-time JSON
 - COPY INTO for batch CSV
-

2 Processing Layer

- Raw tables
 - Streams for CDC
 - Tasks for incremental load
 - MERGE into curated tables
-

3 Performance

- Natural clustering on date
 - Dedicated warehouses (ETL, BI)
 - Auto-suspend enabled
 - Multi-cluster for BI
-

4 Security

- RBAC design
 - Row access policy by region
 - Mask salary using masking policy
 - Tag sensitive columns
-

5 Governance

- Resource Monitor
 - Access History auditing
 - Object dependency tracking
-

6 Data Protection

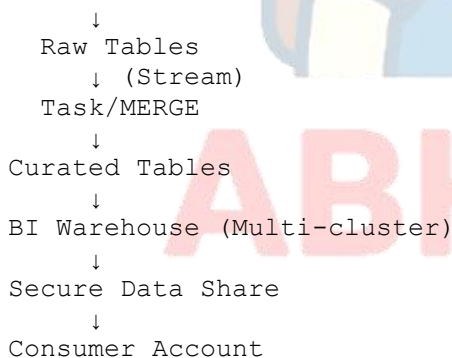
- 7-day retention
 - Clone before deployments
 - Cross-region replication
-

7 Data Sharing

- Share curated sales data to partner account
 - Restrict access using roles
-

10.12 Final Architecture Diagram (Conceptual)

External Files → Stage → Snowpipe/COPY



This is production-ready architecture.

10.13 Capstone Practical Tasks

1. Create full role hierarchy.
2. Load batch CSV.
3. Implement Snowpipe simulation.

4. Build stream + task pipeline.
 5. Apply row & masking policy.
 6. Configure resource monitor.
 7. Clone database before deployment.
 8. Share database with another account.
 9. Design disaster recovery plan.
-

10.14 Interview Mastery Questions

1. Explain secure data sharing in Snowflake.
 2. Difference between sharing and replication?
 3. How does Snowflake avoid data duplication?
 4. Design enterprise-grade Snowflake architecture.
 5. How would you ensure disaster recovery?
 6. How do you balance performance and cost?
 7. Explain full pipeline from ingestion to sharing.
-



Final Summary of Entire Journey

You now understand:

- ✓ Architecture
- ✓ Performance optimization
- ✓ Security & governance
- ✓ Data protection
- ✓ Bulk ingestion
- ✓ CDC pipelines
- ✓ Programming features
- ✓ Monitoring & cost control
- ✓ Data sharing & replication
- ✓ Enterprise architecture design

You are now prepared for:

- Snowflake Developer role
- Snowflake Engineer role
- Production-level implementation
- Technical interviews